

Журналирование БД SQLite

Журналирование изменений БД заключается в последовательной записи во внешнюю память (журнал) всех изменений, выполняемых в базе данных, с целью обеспечить возможность восстановить зафиксированное состояние БД после любого сбоя.

Без журнала [SQLite](#) не сможет откатить незавершенную транзакцию, и если в середине транзакции произойдет сбой или отключение питания, вся база данных будет повреждена.

SQLite предоставляет два режима журналирования БД:

- [Журнал отката](#) (Rollback Journal) - режим, в котором модификация данных производится непосредственно в файле БД, а журнал хранит копии модифицируемых файлов для восстановления в случае отката транзакции или падения приложения.
- [Журнал опережающей записи](#) (WAL) - режим, в котором изменения записываются в основной файл журнала, в то время как файл БД продолжает хранить исходные данные. Вспомогательный файл используется для координации одновременного доступа к БД разными клиентами.

Преимущества и недостатки WAL

WAL имеет три основных **преимущества** перед Журналом отката:

1. Более высокая скорость записи данных.
2. Большой параллелизм. Если изменяемые данные записываются в файл журнала, а не в основной файл БД, читающие транзакции не блокируют пишущие и наоборот.
3. В режиме синхронизации **NORMAL** журналирование WAL гарантирует целостность БД при потере питания.

Однако, у WAL есть ряд **недостатков**:

1. В отличие от Журнала отката, WAL неработоспособен на сетевых файловых системах.
2. Транзакции, включающие изменения в нескольких подключенных БД, неатомарны. Это значит, что в случае падения во время фиксации к части БД изменения могут примениться, а к остальным – нет.
3. [Возобновляемое массовое обновление \(RBU\)](#), оптимизирующее работу с массивными изменениями БД, не поддерживает WAL-журналирование.
4. Добавляется автоматическая операция выгрузки данных из журнала в основной файл БД, что может ухудшить производительность транзакций.

Следует отметить, что в новых версиях SQLite режим WAL избавлен от двух недостатков его ранних реализаций: теперь он поддерживает открытие БД в режиме "только для чтения" и больше не уступает в производительности Журналу отката на крупных транзакциях.

Особенности работы с WAL

При работе БД в режиме WAL следует учитывать следующие аспекты:

- Для обеспечения согласованности одновременного доступа "читателей" и "писателей" к БД в WAL-журналировании используется изолирование «слепок» БД для читающих транзакций. "Читатель" в рамках транзакции продолжает обзирать неизменный «слепок» файла БД таким, какой он был на момент начала читающей транзакции. Изменения пишущих транзакции, фиксируемые до окончания читающей транзакции, невидимы для неё.
- В режиме синхронизации NORMAL при потере питания или падении ОС возможен откат завершенной транзакции. Тем не менее, если стойкость БД не является необходимой характеристикой, рекомендуется с WAL-журналированием использовать именно NORMAL-синхронизацию из-за значительного преимущества фиксации транзакций перед FULL-синхронизацией.
- После перехода к WAL-журналированию нельзя изменить размер страницы данных, как для пустой БД, так и при выполнении команды VACUUM и при восстановлении из резервной копии. А также нельзя изменить режим блокировки (PRAGMA schema.locking_mode), если до перехода к WAL-журналированию он был выставлен в EXCLUSIVE. По исполнению команды VACUUM следует разрывать и переоткрывать все соединения с БД.

Сравнение производительности режимов журналирования

При работе с транзакциями в SQLite возник вопрос выбора режима журналирования, необходимого для обеспечения возможности восстановить данные БД после любого сбоя. Рассматривались два варианта: **Журнал опережающей записи (WAL)** и **Журнал отката (Rollback Journal)**. Для выявления наиболее оптимального режима журналирования было проведено тестирование.

Методика тестирования

Тестирование производилось на **Wasaby Framework 21.2000**, включающей [SQLite 3.33.0](#). Все вызовы в БД осуществлялись через публичный API модуля [sbis-dbi](#). В качестве тестового стенда использовался ПК под управлением Windows 10 20H2, Intel Core i7 3.00GHz, 32GB RAM, SSD.

Тестовая таблица **journaling_performance** имела следующий формат:

- key_field: INTEGER PRIMARY KEY
- integer_field: INTEGER
- real_field: REAL
- text_field: TEXT
- blob_field: BLOB

Тестовые сценарии:

- Вставка данных группами по 1/10/100/1000 записей в транзакции:

```
INSERT INTO journaling_performance ( key_field, integer_field, real_field, text_field, blob_field ) VALUES ( $1, $2, $3, $4, $5 )
```
- Обновление данных группами по 1/10/100/1000 записей в транзакции:

```
UPDATE journaling_performance SET integer_field = $2, real_field = $3, text_field = $4, blob_field = $5 WHERE key_field = $1
```
- Удаление данных группами по 1/10/100/1000 записей в транзакции:

```
DELETE FROM journaling_performance WHERE key_field = $1
```
- Выборка данных группами по 1/10/100/1000 записей в читающей транзакции:

```
SELECT * FROM journaling_performance WHERE key_field = $1
```
- Выборка данных:

```
SELECT * FROM journaling_performance WHERE key_field = $1
```

Результаты тестирования

В таблице ниже приведены лучшие результаты среди серии прогонов тестовых сценариев.

	Журнал упреждающей записи, запросов в секунду	Журнал отката, запросов в секунду
Insert (1 запись в транзакции)	750,955	137,939
Insert (10 записей в транзакции)	3177,85	1019,46
Insert (100 записей в транзакции)	4448.99	3343.43
Insert (1000 записей в транзакции)	4619,39	4560,47

Update (1 запись в транзакции)	754,35	137,135
Update (10 записей в транзакции)	3208,42	1022,43
Update (100 записей в транзакции)	4390.45	3230.15
Update (1000 записей в транзакции)	4627,93	4342,83
Delete (1 запись в транзакции)	737,13	135,255
Delete (10 записей в транзакции)	3439,95	1029,86
Delete (100 записей в транзакции)	5045.47	3662.72
Delete (1000 записей в транзакции)	5387,64	5224,54
Select (1 запись в транзакции)	1216,31	1165,33
Select (10 записей в транзакции)	3070,62	2965,71
Select (100 записей в транзакции)	3273.45	3277.74
Select (1000 записей в транзакции)	3387,55	3356,88
Select (без транзакции)	3600,79	3198,15

Выводы

В результате сделаем вывод, что режим **WAL** демонстрирует значительное преимущество в быстродействии операций изменения БД для небольших транзакций и не уступает **Журналу отката** в крупных. Показывает аналогичное быстродействие выборки данных в читающих транзакциях и серьезное преимущество выборки вне транзакции.

При выборе режима журналирования БД SQLite следует руководствоваться ограничениями WAL. Следовательно, в рамках приложения SBIS при выборе режима журналирования БД **SQLite** следует учитывать, что если сценарий применения БД требует её разворота на сетевых файловых системах, обязывает вносить изменения в несколько подключенных БД в рамках одной транзакции или предполагает использование возобновляемого массового обновления, нужно выбирать **Журнал отката**. Во всех остальных сценариях рекомендуется использовать **WAL**.

Использование WAL в Wasaby Framework

SQLite не предоставляет возможность создать БД сразу в режиме WAL, метод журналирования можно изменить только у готовой БД.

Для переключения БД в режим журналирования WAL достаточно выполнить запрос **PRAGMA journal_mode = WAL**:

- На C++:

```
sbis::SqlQuery( L"PRAGMA journal_mode = WAL;" );
```

- На Python:

```
sbis.SqlQuery( 'PRAGMA journal_mode = WAL;' )
```

Запрос на переключение желательно делать во время инициализации соединения с БД, когда доступ к БД идёт только из потока инициализации. Если нет возможности гарантировать эксклюзивность доступа к БД, достаточно проверять возвращаемое запросом значение. SQLite вернёт исходный режим журналирования, если переключение в текущий момент не возможно (к примеру, если в БД есть активная транзакция).

Как скачать БД из мобильного приложения

Для того, чтобы скачать БД в режиме WAL из мобильного приложения и открыть ее на ПК, нужно выгрузить 3 файла:

- Основной файл базы данных - содержит полное состояние базы данных SQLite.
- Файл журнала базы данных (файл в формате .wal) - журнал с повтором транзакций, в котором записываются транзакции, которые были зафиксированы, но еще не применены к основной базе данных;

- Файл WAL-index (файл в формате .shm) - используется для координации доступа к базе данных для нескольких клиентов, а также в качестве кэша, помогающего клиентам быстро находить фреймы в файле .wal.